

Efficient and retargetable SIMD translation in a dynamic binary translator

Sheng-Yu Fu¹  | Ding-Yong Hong² | Yu-Ping Liu¹ | Jan-Jan Wu² | Wei-Chung Hsu¹

¹National Taiwan University, Taipei, Taiwan

²Academia Sinica, Taipei, Taiwan

Correspondence

Sheng-Yu Fu, National Taiwan University, Taipei 10617, Taiwan.
Email: d03922013@csie.ntu.edu.tw

Funding information

Ministry of Science and Technology, Taiwan, Grant/Award Number: MOST 105-2622-8-002-002 and MOST-106-2218-E-002-040

Summary

The single-instruction multiple-data (SIMD) computing capability of modern processors is continually improved to deliver ever better performance and power efficiency. For example, Intel has increased SIMD register lengths from 128 bits in streaming SIMD extension to 512 bits in AVX-512. The ARM scalable vector extension supports SIMD register length up to 2048 bits and includes predicated instructions. However, SIMD instruction translation in dynamic binary translation has not received similar attention. For example, the widely used QEMU emulates guest SIMD instructions with a sequence of scalar instructions, even when the host machines have relevant SIMD instructions. This leaves significant potential for performance enhancement. We propose a newly designed SIMD translation framework for dynamic binary translation, which takes advantage of the host's SIMD capabilities. The proposed framework has been built in HQEMU, an enhanced QEMU with a separate thread for applying LLVM optimizations. The current prototype supports ARMv7, ARMv8, and IA32 guests on the X86-64 AVX-2 host. Compared with the scalar-translation version HQEMU, our framework runs up to 1.84 times faster on Standard Performance Evaluation Corporation 2006 CFP benchmarks and up to 6.81 times faster on selected real applications.

KEYWORDS

AVX, dynamic binary translation, helper function, LLVM, NEON, QEMU, single instruction multiple data

1 | INTRODUCTION

Single instruction multiple data (SIMD) instructions compute multiple data items in a single cycle. We illustrate SIMD instruction semantics and the comparison with scalar instructions in Figure 1. The execution of SIMD instructions avoids the overhead of redundant instruction fetching, decoding, data dependency checking, and writing results back to registers. Therefore, SIMD is both power and execution efficient. The SIMD instructions have been widely supported in modern microprocessors, such as streaming SIMD extension (SSE)/advanced vector extension (AVX)^{1,2} for X86, NEON^{3,4} for ARM, scalable vector extension⁵ for ARMv8, AltiVec^{6,7} for PowerPC, and MIPS SIMD Architecture⁸ for MIPS. At the same time, SIMD code generation in compilers has also made significant progress such as superword-level parallelism,⁹⁻¹¹ polyhedral model autovectorization,¹² whole function vectorization,¹³ outerloop vectorization,¹⁴ autovectorization for noncontiguous memory access patterns,¹⁵⁻¹⁹ partial vectorization,²⁰ optimization for SIMD permutation operation,²¹ and alignment

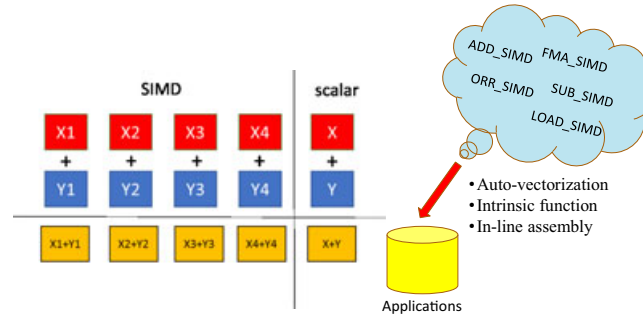


FIGURE 1 Single-instruction multiple-data (SIMD) instruction [Colour figure can be viewed at wileyonlinelibrary.com]

handling,^{22,23} just to name some recent advances. We expect to see a greater proportion of SIMD instructions used in current and future applications.

However, many retargetable dynamic binary translator (DBT) do not address the translation of SIMD instructions. For example, in the widely used QEMU,²⁴ guest SIMD instructions are emulated by helper functions. To simplify the translation of complex guest SIMD instructions and ease porting to various host industry standard architectures (ISAs), these helper functions are implemented with high-level language (eg, C or C++). For easy porting, helper functions are translated into sequences of scalar instructions, even when the host machine is equipped with a SIMD instruction set. Invoking a helper function pays a function call overhead. In addition, since the translator cannot be sure whether the helper function reads or modifies any guest states, the emulated guest states need to be saved to memory before the helper function is called. With the aforementioned SIMD translation scheme, there are significant overheads, including function calling, redundant guest state synchronization, and inefficient instruction emulation. Such inefficiencies leave a significant performance issue for DBTs to address.

In this paper, we propose a newly designed SIMD translation framework for DBT. In our framework, the host SIMD capability can be fully utilized when translating guest SIMD instructions. Different ISAs provide support differently for SIMD instructions. Some complex SIMD instructions are only supported by specific architectures. For example, ARM NEON supports byte-indexed table lookup (vector table lookup (VTBL))²⁵ using byte indexes to search byte values in a list of SIMD registers. We illustrate the functionality of VTBL in Figure 2. However, X86-based machines do not support VTBL instructions. When migrating an ARM NEON executable to X86 platforms, efficiently translating an ARM NEON instruction into X86 instructions is a challenge. To overcome such challenges, we combine the advantages of both vector instruction registers (IRs) and helper functions. In our system, commonly-used SIMD instructions are translated to vector IRs. Complex ISA-specific SIMD instructions are still emulated by helper functions, but, in order to eliminate the overheads of helper function call and redundant guest state synchronization, these helper functions will be inlined. We have implemented the SIMD translation framework in hybrid QEMU (HQEMU).²⁶ Currently, our system can support ARMv7, ARMv8, and IA32 guests and run on the X86-64 AVX2 host. In summary, this work makes the following contributions.

1. We propose a hybrid design for efficient and retargetable SIMD translation for DBT. This framework combines the advantages of vector IRs and helper function inlining to achieve efficient cross-ISA SIMD translation.
2. We evaluate our framework with Standard Performance Evaluation Corporation (SPEC) 2006 CFP and a set of real-world applications with a variety of guest ISAs on the X86-64 AVX2 host. The baseline used for execution time comparison is scalar-translation version HQEMU. For SPEC 2006 CFP, ARMv8-to-AVX2 translation runs up to 1.84 times

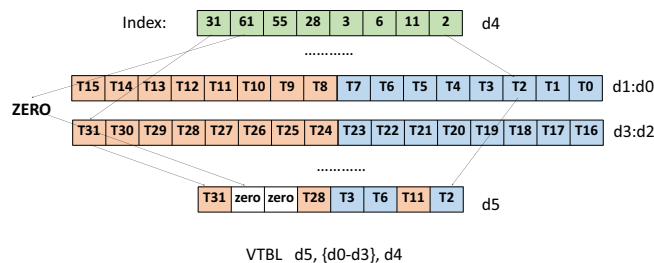


FIGURE 2 Byte indexes table lookup (vector table lookup (VTBL)) [Colour figure can be viewed at wileyonlinelibrary.com]

faster and IA32-to-AVX2 translation runs up to 1.52 times faster. For the selected real applications, ARMv7-to-AVX2 runs up to 6.81 times faster. We believe the proposed SIMD translation framework is applicable to other DBT tools.

The rest of this paper is organized as follows. Section 2 will give the background of this work. Section 3 will detail the design and implementation of our vector IR and helper function inlining mechanisms. Section 4 will evaluate the effectiveness of our SIMD translation framework. Section 5 will discuss related work and Section 6 will summarize and conclude.

2 | BACKGROUND

2.1 | Dynamic binary translation

Dynamic binary translation²⁷ is a common technique used in cross-ISA process virtual machines²⁸ to speed up guest machine emulation. With DBT, QEMU²⁴ is a very successful and widely adopted fast machine emulator. The popular Android emulator is implemented based on QEMU. The Android emulator emulates ARM executables in the Android environment on X86 based platforms. Dynamic binary translators dynamically translate guest executables (eg, ARM) into host (eg, x86) instructions and cache the translated host code in memory to avoid retranslation. The translated host native code runs many times faster than the traditional interpretation approach. However, speeding up the emulation by generating optimized code has been challenging since optimizations require longer translation time, which is part of the runtime, and may lead to unreliable code if the optimizations are complex and not thoroughly tested. Normally, in DBT, short-running applications are preferred for fast code generation while long-running applications require more optimized code.

To benefit both short- and long-running applications, HQEMU²⁶ adopts a multithreaded translation and optimization approach. The original QEMU fast translation path via tiny code generation (TCG) and the newly added optimization thread, which leverages LLVM optimization passes, are operated in parallel. If the application is short running, it might finish execution before the optimized code is generated from the optimization thread. Hence, the optimization time has no impact on the TCG generated code. If the program is long running, the original TCG generated code could be replaced by redirecting the execution to the optimized code generated by LLVM.

HQEMU²⁶ is a 2-level IR based DBT. The first level IR is the original TCG IR. The TCG IR is converted to LLVM IR by the optimization thread and fed into LLVM optimization passes. Figure 3 illustrates the architecture of HQEMU. This approach effectively combines the fast code translation in the TCG of QEMU with the rich optimizations from LLVM to achieve superior performance for both short- and long-running applications. Beside of the performance benefit, 2-level IR also reduces the engineering effort and further increases the system reliability. By leveraging the existing QEMU frontend, HQEMU only needs to translate TCG IRs, which are about 125 different operation codes instead of translating all instructions of guest ISA. For the floating-point support, HQEMU translates guest floating-point instructions to LLVM floating-point operations, which are then translated to host hardware floating-point instructions. The floating-point exceptions are handled properly by the host hardware floating-point units. As reported in the work of

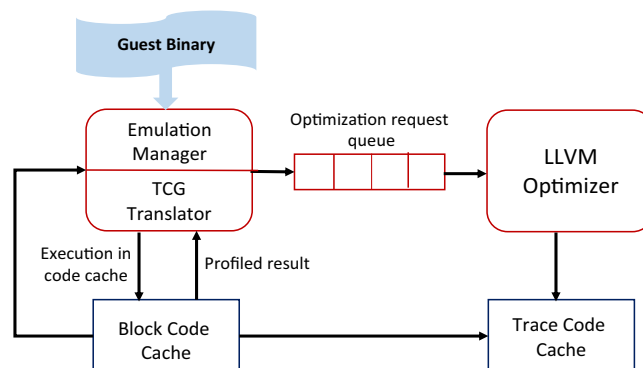


FIGURE 3 The architecture of the hybrid QEMU framework. TCG, tiny code generation [Colour figure can be viewed at [wileyonlinelibrary.com](https://onlinelibrary.wiley.com)]

Hong et al.,²⁶ HQEMU runs SPEC 2006 benchmarks 2.4 to 4 times faster than the original QEMU for emulating X86-32 executables on X86-64 machines.

2.2 | Guest register mapping

Guest register state emulation is one of the primary tasks of binary translation. QEMU maps all guest registers to host memory. However, once access to a guest register is needed, load/store operations must be used. This is inefficient since accessing memory is slower than accessing registers. Mapping the guest registers to a subset of the host registers is therefore a major optimization in modern DBTs such as HP's Aries²⁹ and IA-32 execution layer.³⁰ Register mapping in Aries and IA-32 execution layer is simple since both DBTs are based on the Itanium hosts, which are characterized by having a large register file (ie, 128 general purpose registers and 128 floating-point registers). It is trivial to map each guest register onto 1 unique host register during the entire emulation. However, it is not the case here because the guest of our DBT system may have more SIMD registers than the host. For example, ARMv8 has 32 NEON registers, but X86-AVX2 only has 16 YMM registers. We must analyze the liveness of guest SIMD registers and map rarely used registers to memory. Where a guest register should be mapped is determined by its definition-use chains. The required processing seems complicated. Fortunately, we can leverage the register allocator in LLVM to perform these analyses. We map guest SIMD registers to the LLVM local variables, and apply `PromoteMemoryToRegister` optimization to promote local variables to LLVM virtual registers. Once the guest SIMD registers are promoted, the mapping from guest SIMD registers to host physical registers or memory can be done by the LLVM register allocator. Such a mechanism transforms the guest register mapping problem in retargetable DBTs to compiler register allocation problem and leveraging the register allocation techniques.

2.3 | ARMv7, ARMv8, IA32, and X86-AVX2 SIMD extension

The SIMD architecture varies from one ISA to another ISA. This section lists basic SIMD information of ARMv7, ARMv8, IA32, and X86-AVX2, as shown in Table 1. The information includes SIMD register number, SIMD register width, support for double-precision floating-point operations, and operand number. Compared with ARMv7, ARMv8 extends SIMD register number to 32 and supports double-precision floating-pointing operations; Compared with IA32, X86-AVX2 extends SIMD register width to 256-bit and supports 3 operand instructions. This shows that hardware vendors continually improve SIMD capability so how DBT translates guest SIMD instructions and leverages host SIMD architecture becomes a urgent issue.

3 | SIMD CODE TRANSLATION

The current QEMU TCG only supports about 125 different operation codes, much smaller than most existing ISAs. Therefore, many complex instructions are translated to helper function calls rather than TCG IRs. Helper functions are usually implemented with high-level languages (eg, C or C++), which simplifies implementing complex ISA semantics and porting to various hosts. For example, QEMU TCG uses helper functions to emulate X87 floating point operations, computation of X86 EFLAGS, and SIMD operations.

Figure 4 shows a SIMD translation example with QEMU helper functions. The source code shown in Figure 4A performs a vector addition of 4 single-precision floating-point values. The instructions, ie, `addps` and `vadd.f32`, are generated for X86 SSE and ARM NEON, and their translated code are shown in Figures 4B and 4C, respectively. For the X86 guest, the instruction `addps` is translated to 1 helper function call. In order to simplify porting, QEMU implements these helper functions in scalar code. The helper functions sequentially iterate over each vector element and conduct scalar

TABLE 1 Information of single-instruction multiple-data (SIMD) architectures

	SIMD register number	SIMD register width	Double-precision floating-point operation	Operand number
ARMv7	16	128-bit	No	3
ARMv8	32	128-bit	Yes	3
IA32	8	128-bit	Yes	2
X86-AVX2	16	256-bit	Yes	3

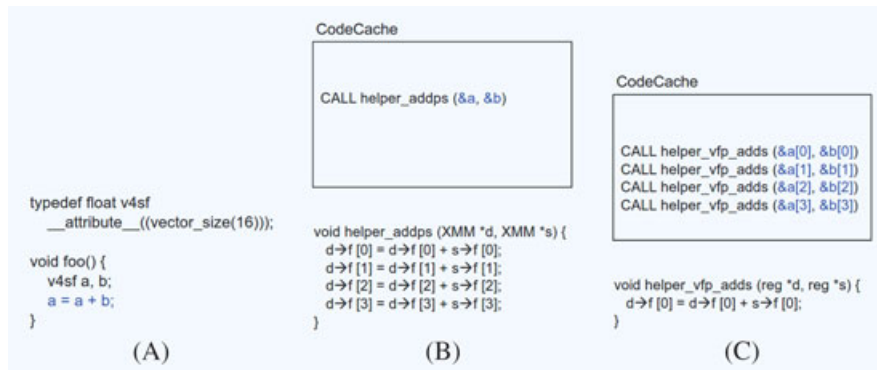


FIGURE 4 A translation example of a single-instruction multiple-data (SIMD) floating-point addition instruction. A, source code; B, x86 streaming SIMD extension emulation; C, ARM NEON emulation [Colour figure can be viewed at wileyonlinelibrary.com]

floating-point addition for each computation. In comparison to the X86 guest, the helper function of the ARM guest is implemented with only 1 scalar floating-point addition. Hence, 4 helper function calls are emitted in the code cache to complete the whole vector operation. This design takes advantage of helper functions but results in 2 performance issues. (1) Guest SIMD instructions are always emulated by scalar instructions even when the host machines support SIMD instructions. (2) The emulated guest states need to be saved to memory before calling a helper function because we cannot ensure the helper functions do not read guest states. Similarly, when returning from a helper function, the guest states must be reloaded to host registers. Such guest state synchronization cost is high when emulating SIMD instructions since the SIMD helper functions are usually small and do not involve lots of computation. Thus, state synchronization often dominates the simulation time of guest SIMD instructions.

To enhance the SIMD translation capability in retargetable DBTs, we propose to enhance the DBT with a well-designed vector IR representing the semantics of guest SIMD instructions. With such an IR layer, we can further apply optimizations^{31,32} and decouple the guest and host to achieve better retargetability. However, some SIMD instructions are ISA-specific and with complex semantics. If we map every SIMD instruction to a specific IR, the IR set would become very large and lose generality. To address this issue, we only translate a set of core SIMD instructions to vector IRs instead of all guest SIMD instructions and remaining SIMD instructions are still translated to helper function calls. However, for those helper function calls, we develop heuristic schemes to guide inlining to minimize excessive calling overhead. Such a hybrid design can achieve high performance but without losing the IR-level generality. However, both vector IR design and helper function inlining mechanisms are not trivial. Vector IRs are used to pass information between different layers. The design of vector IR has an impact on SIMD optimization passes and code generation. On the other hand, helper functions are implemented with C/C++ code and compiled to the host binary at static time. Once the helper functions are translated to host code, the register mapping is already determined. Therefore, we still need to save the register state to memory to synchronize 2 different register mapping configurations, even if we inlined the helper functions. To avoid this limitation, we design a new infrastructure for compiling helper functions and will show the implementation details in this section.

3.1 | Vector support in TCG IR

Our vector IRs are added to TCG, which is the first level IR of HQEMU. We call this vector IR as TCG vector IR. Guest SIMD instructions are represented directly with TCG vector IRs. The example in Figure 4 is translated to the same TCG vector IR (ie, *vadd.f32.128*) for both guest ISAs. Moreover, the TCG vector IRs can be explicitly converted to LLVM vector IRs with the 2-level IR conversion approach in HQEMU and then leverage the rich LLVM optimizations.

An ill-designed IR may impact the translation time, optimization time, and even the host code quality in DBT. For example, if memory access IRs cannot carry alignment information, the code generator either generates aligned memory-access instructions together with runtime checking or generates unaligned memory-access instructions. The considerations about adding portable DBT vector IRs are discussed in this section.

3.1.1 | Vector IR forms and in-memory structure

Instruction registers are used to pass the information between different layers and serve as an interface among compilation passes. Therefore, the form of IR and its in-memory structure vary according to their usage. In general, the IR form can be classified into hierarchical and linear.³³ Hierarchical IR is closer to programming languages and allows nested structures. With nested structures, the control flows, arithmetic operations, function prototypes, and data structures of the source program can be straightforwardly represented. Therefore, hierarchical IRs are implemented with tree³⁴ or graph-based^{35,36} structures.

Linear IR is closer to machine code and thus lower in level than an hierarchical IR. The control transfer statements such as if-then-else, switch, and function calls are translated to explicit machine instructions such as branch, jump, call, and return. Compared with the hierarchical IR, the linear IR carries less information but can be implemented with simple data structures such as array, stack, and linked-list. Traversing hierarchical IR involves processing graphs or tree structures and thus is usually more time-consuming than linear IR traversal. In our DBT system, we consider the following design issues.

- **Vector IR in 2-level IR DBT:** Our vector IR is an extension of TCG IR (it is also the extension of the first-level IR of HQEMU). The first-level IR should facilitate fast translation and perform lightweight optimizations. In this case, array and linked-list are more suitable solutions.
- **The scope in dynamic binary translation:** Since the input of a DBT is guest binary, source program information, such as type of data structures, loop structures, and memory access patterns are missing. In addition, the optimization scope in a DBT is limited to several basic blocks though the granularity can be increased by forming traces. However, even with traces, the optimization scope seen by a DBT is much smaller than the scope (eg, loops, functions, and procedures) visible to a compiler or static binary translators. Therefore, the information that can be obtained from binary is less than from source code. In this case, linear IR is sufficient for a DBT.

Therefore, we implement our vector IR in linear form with an array data structure to favor fast translation. Our design also takes care of extensibility because the SIMD architectures are continuously evolving. For example, Intel proposes FlexVec³⁷ architecture (currently evaluated by the simulator), which supports patch up code generation to enable partial vector code execution. Moreover, ARM plans to enhance their SIMD instruction set with scalable vector extension.⁵ Our vector IR in-memory representation supports variable-number and pointer-type operands. Such design provides flexibility to support future SIMD instructions.

3.1.2 | Compared with LLVM IR

LLVM introduces a set of vector IRs to support vectorization and SIMD code generation. LLVM uses data type to distinguish vector IRs from scalar IRs. Consider the following **SUB** instructions.

reg0 = sub <4 x i32> reg1, reg2. This IR presents a vector instruction, which finishes 4 32-bit integer subtractions by a single instruction.

reg0 = sub <i32> reg1, reg2. This IR presents a scalar instruction, which performs a 32-bit integer subtraction.

reg0 = sub <i64> reg1, reg2. This IR presents a scalar instruction, which performs a 64-bit integer subtraction.

To conform to QEMU TCG IRs, directly embedding the operations into opcode, our vector IRs follow the similar philosophy so this LLVM IR **reg0 = sub <4 x i32> reg1, reg2** is equivalent to our TCG vector IR **reg0 = sub_4_i32 reg1, reg2**.

3.1.3 | Compound vector IR support

A compound instruction can perform multiple operations within 1 instruction. For example, the ARMv8 NEON instruction *MLA* (multiply-add to accumulator) can perform an addition followed by a multiplication in 1 instruction. There are also studies about how to design a high performance and power efficient processor based on compound instructions.³⁸ In our vector IR design, we consider supporting compound instructions in the vector IR. We consider whether the instruction *MLA* should be translated to 1 multiply-add IR instruction or converted to 2 IR instructions, 1 multiplication plus 1 addition instruction. If supporting compound vector IRs, the variety of different guest SIMD instructions would increase the amount of vector IR to be designed in TCG. However, in our 2-level IR DBT, once the information is lost in the first-level IR, it is difficult and time consuming to recover in later levels. Although the optimizer could try to recognize the patterns, which are composed of multiplication and an addition, pattern matching is time consuming. Therefore, it is not suitable

TABLE 2 Opcodes Supported in tiny code generation vector instruction register

Operation	Instruction example
Logic operation	vand_128, vand_64, vorr_128, vorr_64, vorn_128, vorn_64
Integer arithmetic	vadd_i8_128, vadd_i16_128, vadd_i32_128, vadd_i64_128, vadd_i8_64, vsub_i8_128
Floating arithmetic	vmul_f32_128, vfma_f32_128, vmls_f32_128
Bit manipulation	Bitwise_Select_128, Bitwise_Insert_If_True_128, Bitwise_Insert_If_False_128
Load, store, and data movement	vload_128, vstore_128, vmove_128
Pack/unpack	Unpack_low_128, Unpack_high_128
Type conversion	Float_to_integer_f32_128, Integer_to_float_f32_128

for a DBT system. Thus, compound vector IR support is more helpful to DBT systems. Although HQEMU performs the optimizations on another thread and avoids long optimization time delays, HQEMU can still benefit from compound vector IR. The reason is that if the optimized code is generated early, the emulation can benefit by redirecting the execution to optimized code early.

3.1.4 | Opcodes supported in TCG vector IR

Currently, we implement 86 vector operations by analyzing the 2 most popular ISAs, ie, ARM and X86, and profiling a broad class of applications, including scientific, multimedia, and other popular benchmarks. The set of vector IR instructions we have implemented cover data processing (ie, arithmetic and logical computation instructions), data movement, bit manipulation, pack/unpack, type conversion, and vector load/store instructions. Because some operations such as logical operations only have integer type, we separately count floating-point operations and integer operations. For example, `vadd_f32` and `vadd_i32` are viewed as 2 different operations among 86 supported vector operations. `vadd_i32` and `vadd_i8` are all integer operations so they are viewed as the same operation.

Table 2 lists the type of vector IR supported. In Table 2, the vector instruction format is of `Opcode_<ElementType>_<VectorLength>`. For example, instruction `vorn_128` conducts bitwise OR-NOT on 128-bit vectors (the element type is omitted with logical operations); `vadd_i8_128` performs 16 8-bit integer additions at the same time, and instruction `vfma_f32_128` performs multiply-and-accumulate operation on 4 single-precision floating-point values.

Bit manipulation operations are important when converting control flow to data flow. We implement 2 common bit manipulation operations, ie, `Bitwise_Select` and `Bitwise_Insert_If_True`. They all have 1 destination register and 2 source registers. `Bitwise_Select` selects the *i*th bit from the first source register if the *i*th bit of the destination register is 1; otherwise, it selects the *i*th bit from the second source register. `Bitwise_Insert_If_True` selects the *i*th bit from the first source register if the *i*th bit of the second source register is 1. If the *i*th bit of the second register is 0, `Bitwise_Insert_If_True` leaves the *i*th bit of the destination register unchanged.

3.2 | Selective SIMD related helper functions inlining

In current QEMU, helper functions are implemented with C code and compiled to host binary before execution. Compiling helper functions in advance instead of at run time can reduce the host code generation time. However, once the helper functions are translated to host binary, the register mapping is already determined and results in 2 different register mapping configurations. One configuration is for the trace where the SIMD instruction is in and another is for the translated helper functions. Therefore, the load/store instructions, which are used to synchronize 2 register mapping configurations, become indispensable even if we inline the binary version helper functions.

Since QEMU maps guest register states to memory, all reading and writing to registers are translated to load/store instructions, as shown in Figure 5. Figure 5 shows the translation flow of `vadd vreg0 vreg1 vreg2`, which is a vector add instruction. The registers `vreg1` and `vreg2` are first loaded to temporary registers, and the result of addition is written back to `vreg0`. Because the guest register state is written back to memory when instructions are simulated, QEMU does not need to generate extra loads/stores to synchronize the guest register state between the trace and the helper functions.

However, if we map guest registers to host registers, we have to generate store instructions for writing back the guest register states before leaving the trace. Moreover, after returning to the trace, we need to generate load instructions for

```

vadd vreg0 vreg1 vreg2
    ↓ TCG Translator
1. vload  vtmp1 *cpu_vreg1
2. vload  vtmp2 *cpu_vreg2
3. vadd   vtmp0 vtmp1 vtmp2
4. vstore vtmp0 *cpu_vreg0

```

FIGURE 5 vadd vreg0 vreg1 vreg2 QEMU translation flow [Colour figure can be viewed at wileyonlinelibrary.com]

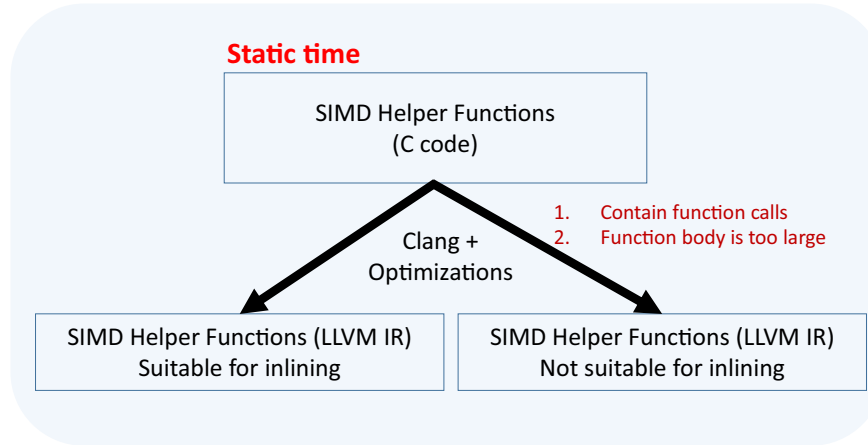


FIGURE 6 Compilation of helper functions. SIMD, single instruction multiple data. IR, instruction register [Colour figure can be viewed at wileyonlinelibrary.com]

loading guest register states to host registers. Such register synchronization overhead is overwhelming when the guest SIMD instructions are translated to multiple helper functions or the guest-ISA has many registers. To solve this problem, we have to redesign the compilation infrastructure of helper functions.

Similarly to QEMU, we translate and optimize helper functions before run time. However, we translate helper function to LLVM IRs instead of host binary. In this stage, we divide the helper functions into 2 categories, ie, suitable for inlining and not suitable for inlining, as shown in Figure 6. If the instruction count is more than a predefined threshold, we do not inline this helper function to prevent severe code bloat that can cause instruction cache thrashing. Another reason to avoid inlining is that the helper function contains function calls and definition-use of guest registers within these function calls is hard to analyze, such as with library calls. For the helper functions, which cannot be inlined, we would analyze the definition-use chain of the function body. Based on the analyzed information, our translator can remove redundant register state synchronizations.

3.3 | Register mapping and redundant state synchronization elimination

HQEMU simulates the guest executable via 2-level IR translations. The first stage is carried out by QEMU TCG. The guest binary is translated to TCG IRs and helper function calls in the this stage, as shown in Figure 5. The second stage translates TCG IRs to LLVM IRs. To leverage the LLVM register allocator, HQEMU maps the guest registers to local variables, as shown in Figure 7. The guest register state should be copied to the local variables when entering the trace. The local variables are copied back to guest register states when exiting the trace. Moreover, the loads/stores to the guest register

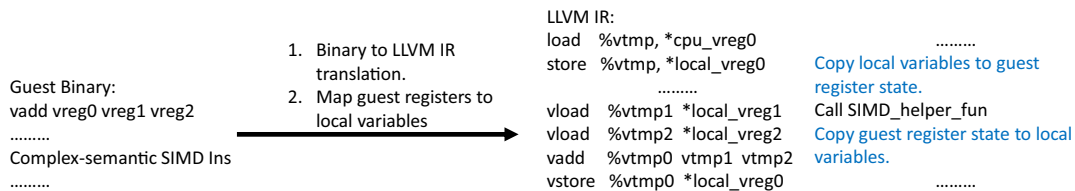


FIGURE 7 Guest binary to LLVM instruction register (IR) translation. SIMD, single instruction multiple data [Colour figure can be viewed at wileyonlinelibrary.com]

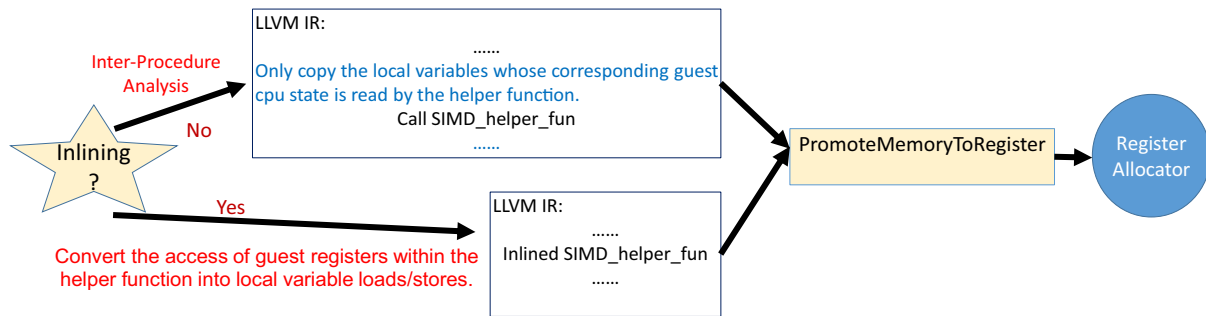


FIGURE 8 Helper function inline and noninline. IR, instruction register [Colour figure can be viewed at wileyonlinelibrary.com]

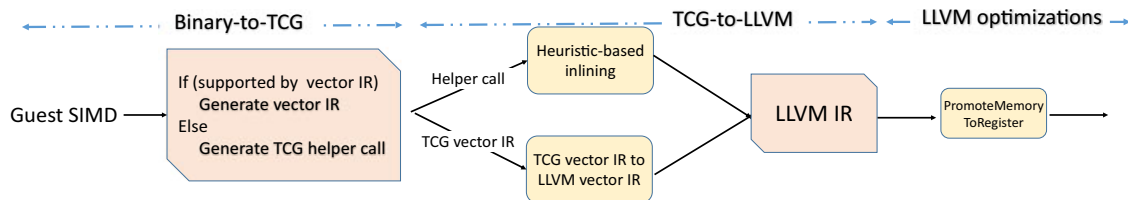


FIGURE 9 Single-instruction multiple-data translation infrastructure. IR, instruction register; TCG, tiny code generation [Colour figure can be viewed at wileyonlinelibrary.com]

state, such as instructions 1, 2, and 4 of Figure 5, are converted to loads/stores of the local variables, as shown in Figure 7. Because helper functions calls are considered as exits of the trace, we should copy local variables to guest register states. To make our design more modular, we decouple the translation and the optimization. We copy all the local variables back to guest register states in the translation phase, and these redundant copies can be eliminated by the later optimization passes.

After the LLVM IR is translated, our SIMD inliner is invoked to determine whether the SIMD helper function should be inlined. Figure 8 shows the decision process to determine whether the helper function should be considered an inline candidate or not.

- **Not to be inlined:** If the helper function is not an inline candidate, we must rely on the definition-use chain and interprocedural analyses to eliminate redundant state synchronizations. We can label the guest registers, which are accessed by the helper function, and remove the synchronizations for other unused registers. Finally, we apply the LLVM PromoteMemoryToRegister optimization to promote the local valuables to LLVM virtual registers and leverage the LLVM register allocator to finish the register mapping.
- **To be inlined:** If the helper function is an inline candidate, it will be inlined. Once it is inlined, we convert the access of guest registers within the helper function to loads/stores of local variables. Once all the references of guest registers within the trace are converted to local variable loads/stores, the LLVM PromoteMemoryToRegister transformation is invoked.

At the final part of this section, we summarize our design in Figure 9. In the first stage (Binary-to-TCG), guest SIMD instructions are translated to TCG vector IRs or TCG helper function calls. Second stage (TCG-to-LLVM) iterates TCG IRs and translates TCG IRs to LLVM IRs. If the TCG IR is a vector IR, an equivalent LLVM IR or equivalent LLVM IR sequence is generated. On the other hand, if the TCG IR is a helper function call, the heuristic-based inliner is invoked. Finally, the second stage outputs LLVM IRs whether the TCG IRs coming from the first stage are vector IRs or helper function calls. Once the second stage is finished, we invoke PromoteMemoryToRegister to promote local variables to LLVM virtual registers and therefore the guest registers are mapped to LLVM virtual registers.

4 | EXPERIMENTAL EVALUATION

In this section, we evaluate the effectiveness of the proposed hybrid design for efficient and retargetable SIMD translation. First, we describe the experimental environment and benchmark suite we use. Then, we evaluate the performance impact

of our vector IR support and selective inlining mechanisms. We also discuss the impact of SIMD/scalar instruction ratios on the effectiveness of our optimizations.

4.1 | Experiment environment

We implement the translation from guest binary to our vector IR in the IA32, ARM-v7, and ARM-v8 frontends. These 3 frontends have the same X86-AVX2 backend since our HQEMU is installed on X86-64 machines. Table 3 is the host machine, operating system, and HQEMU version.

The compilers we used for IA-32 applications were gcc-5.3, gfortran-5.3, and g++-5.3; the compilers for ARM-v7 were arm-linux-gnueabi-gcc-5.3, arm-linux-gnueabi-gfortran-5.3, and arm-linux-gnueabi-g++-5.3; the compilers for ARM-v8 were aarch64-linux-gnu-gcc-5.3, aarch64-linux-gnu-gfortran-5.3, and aarch64-linux-gnu-g++-5.3. The vectorization flags used were -ftree-vectorize and -ftree-slp-vectorize. Because ARM-v7 NEON does not support double-precision floating-point operations, both loop-based and superword-level parallelism vectorizer in gcc would output "nonsupport vector type" messages for benchmarks containing double-precision computations. For example, in Figure 10, although only d[i] is a double-precision floating point, gcc still translates statement S1 to all scalar instructions. Therefore, many double-precision benchmarks in SPEC 2006 use only a few NEON instructions.

To demonstrate the strength of our dynamic SIMD translation capability, we select the popular SPEC 2006 CFP benchmark with *reference* input for ARMv8 and IA32 guests. SPEC 2006 CFP benchmark provides 3 kinds of input data which are *test*, *train*, and *reference*. These input data sets are classified according to the size of data set. *Test* data set is the smallest. *Train* data set is the median and *reference* data set is the largest. Because the SPEC 2006 CFP contains many double-precision floating point operations, we selected another benchmark set consisting of FFT³⁹ and TSVC loops⁴⁰ for ARM-v7 guest. FFT is the Fast Fourier transform and TSVC loops are benchmarks for vectorization designed by Callahan, Donagarr, and Levine. In the following sections, we show the improved performance by our SIMD translation framework. If the baseline is not marked, the baseline is the scalar-translation version HQEMU, which translates guest SIMD instructions by scalar helper functions.

TABLE 3 Host machine environment and benchmark

Machine	Intel(R) i7-5930K CPU @ 3.50 GHz
OS	Linux 3.16.5-gentoo
Benchmark set	SPEC 2006 CFP, TSVC, FFT
HQEMU version	Based on official QEMU 2.2.0

Abbreviations: FFT, fast Fourier transform; HQEMU, hybrid QEMU; OS, operating system; SPEC, Standard Performance Evaluation Corporation.

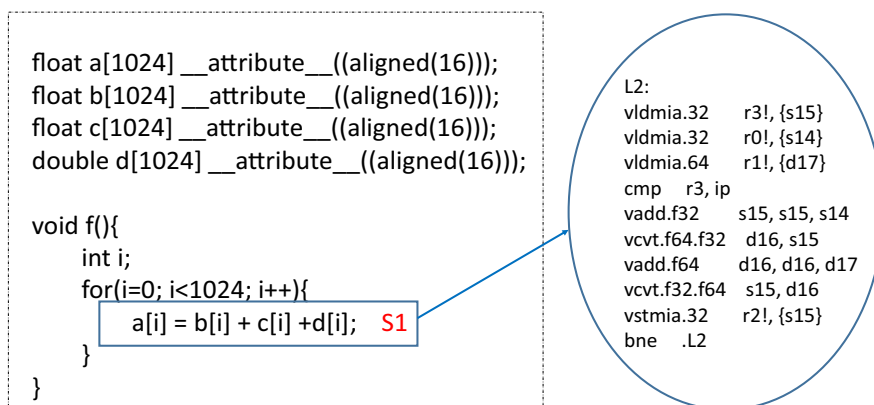


FIGURE 10 S1's binary, which is generated by gcc-5.3. Although only d[i] is a double-precision floating point, gcc-5.3 still translates statement S1 to all scalar instructions [Colour figure can be viewed at wileyonlinelibrary.com]

4.2 | Performance improvement

Figure 11 shows the performance improvement of our hybrid design and Figures 12 and 13 show dynamic instruction distribution of SPEC 2006. In Figures 12 and 13, we categorize all executed instructions into SIMD and scalar instructions and show the respective ratios. Some of SPEC 2006 benchmarks have low SIMD ratios. Without SIMD instructions, our enhancement has little impact. Therefore, Figure 11 does not include those benchmarks with low SIMD ratios. CactusADM and Sphinx3 cause segmentation faults in the official QEMU when the guests are IA32 and ARMv8. Thus,

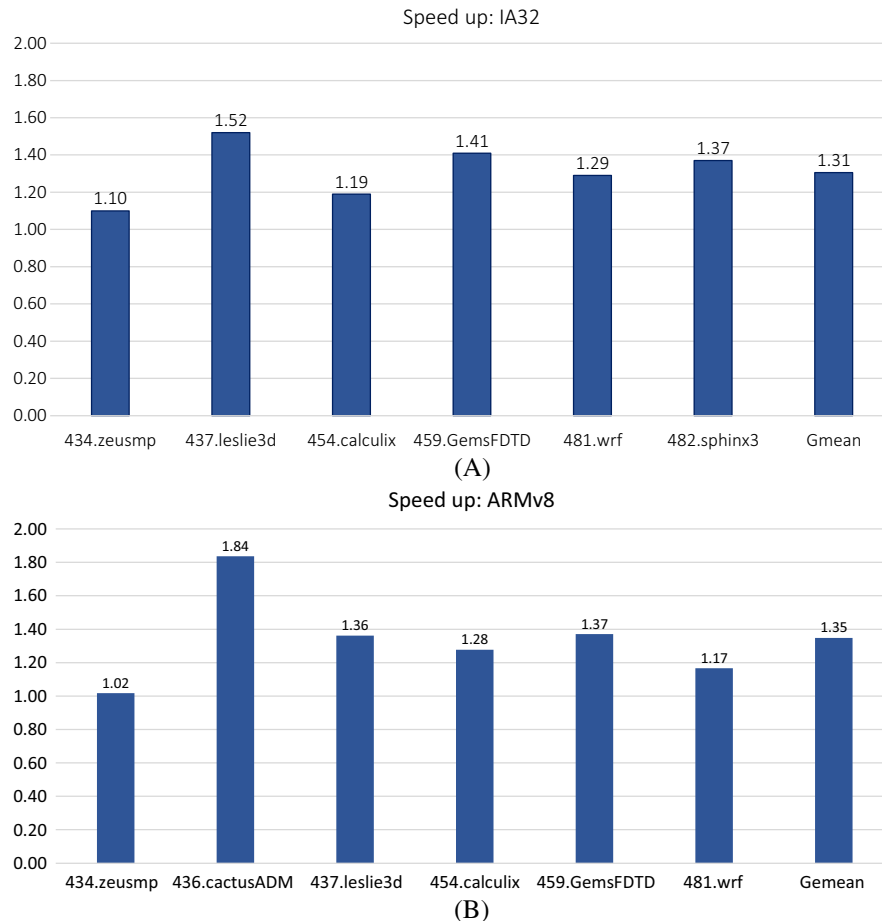


FIGURE 11 Standard Performance Evaluation Corporation 2006 performance improvement. A, IA32 speed up; B, ARMv8 speed up [Colour figure can be viewed at wileyonlinelibrary.com]

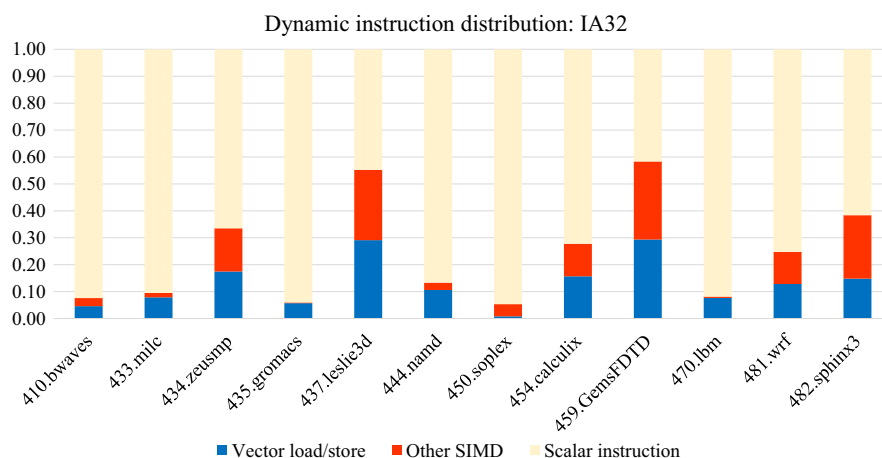


FIGURE 12 Standard Performance Evaluation Corporation 2006 IA32 dynamic instruction distribution [Colour figure can be viewed at wileyonlinelibrary.com]

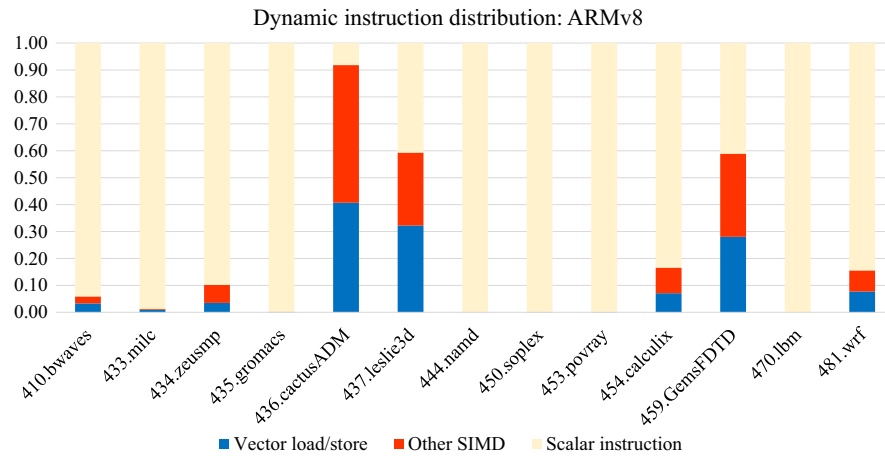


FIGURE 13 Standard Performance Evaluation Corporation 2006 ARMv8 dynamic instruction distribution [Colour figure can be viewed at wileyonlinelibrary.com]

they are also omitted in Figure 11A and 11B. Among these benchmarks, 427.leslie3d, 459.GemsFDTD, and 436.cactusADM have more significant performance improvement than other benchmarks. For example, 427.leslie3d reaches 1.52 times speedup and 459.GemsFDTD reaches 1.41 times speedup when the guest is IA32. 436.cactusADM reaches 1.84 times speedup and 459.GemsFDTD reaches 1.37 times speedup when the guest is ARMv8. This speed improvement is because these benchmarks have higher SIMD ratios than others. Because our approach focuses on efficient SIMD translation, more SIMD instructions in the guest binary, more speedup we can achieve. On SPEC 2006, our translation framework reaches a geometric mean of 1.31 times on IA32 guest and 1.35 times on ARMv8 guest.

Figure 14 shows the vector IR coverage of SPEC 2006. The definition of coverage is $\#(\text{guest SIMD instructions translated to vector IR}) / \#(\text{guest total SIMD instructions})$. For example, 437.leslie3d in Figure 14A has 0.99 coverage means 99%

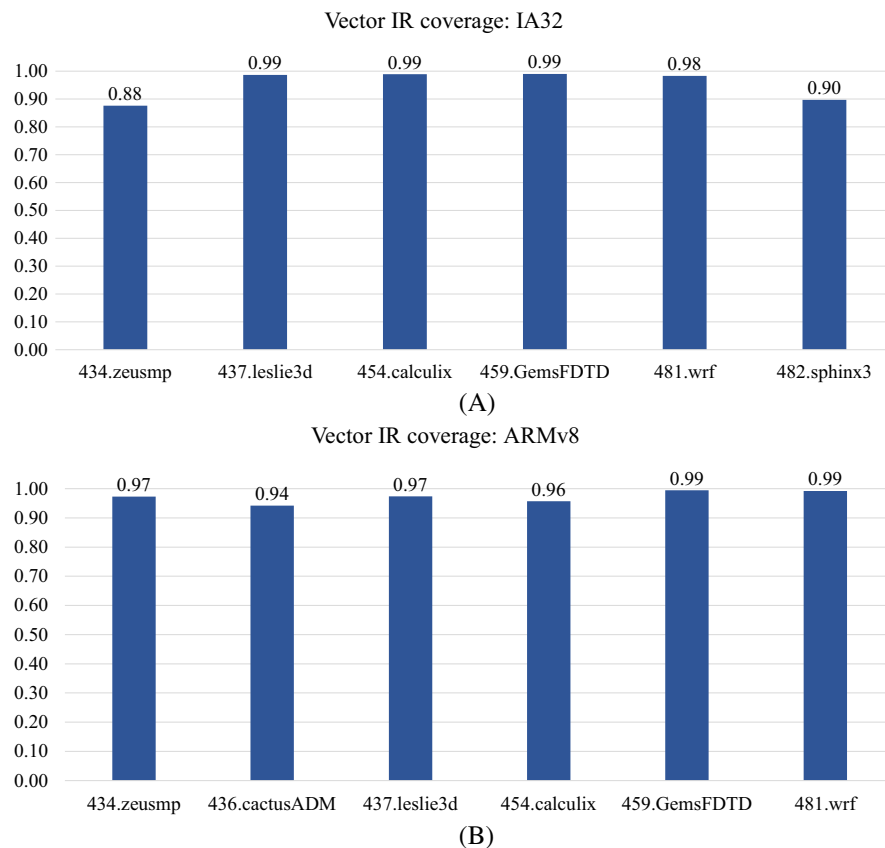


FIGURE 14 Standard Performance Evaluation Corporation 2006 vector instruction register (IR) coverage. A, IA32 vector IR coverage; B, ARMv8 vector IR coverage [Colour figure can be viewed at wileyonlinelibrary.com]

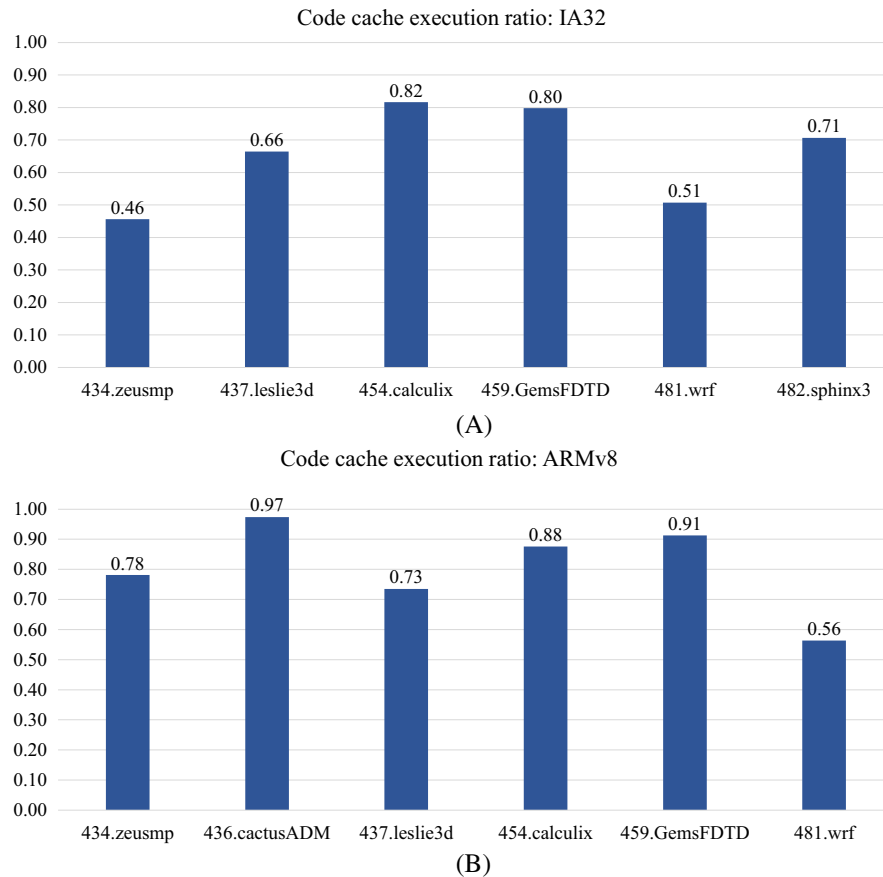


FIGURE 15 Standard Performance Evaluation Corporation 2006 code cache execution time ratio. A, IA32 code cache execution time ratio; B, ARMv8 code cache execution time ratio [Colour figure can be viewed at wileyonlinelibrary.com]

of guest (IA32) SIMD instructions are translated via vector IRs. Figure 15 illustrates the time portion executing in code cache. The definition of code cache execution time portion is $(\text{time in code cache})/(\text{total execution time})$. We can find that high vector IR coverage does not imply high code cache execution time portion. The reason is that register state synchronizations are eliminated when guest SIMD instructions are translated via vector IRs. Therefore, vector IR translation also reduces the execution time in code cache. From Figure 15, we can observe that 481.wrf is relatively low in both guests. The reason is guest scalar instructions emulated by helper functions occupy great execution time portion. 434.zeusmp is only 46% when guest is IA32. The reason is 4% of total executed instructions are unordered compare scalar double-precision floating-point values and set EFLAGS (UCOMISD). The UCOMISD instructions are emulated via helper functions which implemented by software floating-point. Therefore, emulation of UCOMISD instructions occupies significant execution time portion.

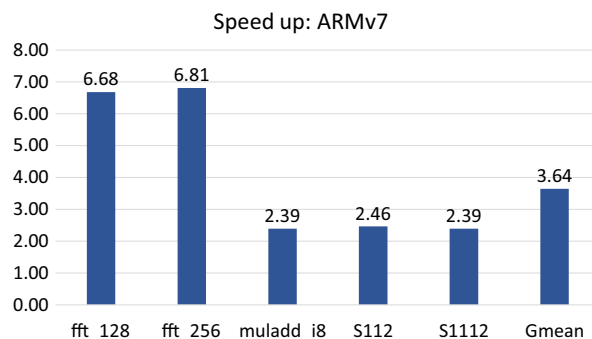


FIGURE 16 ARMv7 speed up [Colour figure can be viewed at wileyonlinelibrary.com]

4.3 | Performance improvement for ARM

Because ARMv7 NEON does not support double-precision operations, when the guest is ARMv7, our framework has limited contributions to make with SPEC 2006. To demonstrate the potential of our translation framework on the ARMv7 guest binaries, we selected a set of real-world benchmarks comprised of FFT for signal processing, matrix multiplication with 8-bit data type for multimedia, and TSVC loops. Figures 16 and 17 show the performance improvement. Our system runs up to 6.81 times faster than the scalar version of HQEMU for ARMv7 guest and 5.84 times faster for ARMv8 guest. Figures 18 to 21 show L1 data cache load/store reduction from our translation framework. The numbers shown in Figures 18 to 21 are the L1 data cache access count, which is normalized to the scalar version of HQEMU.

4.3.1 | Fast Fourier transform

This benchmark performs the FFT and is downloaded from Spiral 6.0.³⁹ Most SIMD instructions in the hot trace are translated to vector IRs, and the vector IRs can be one-to-one or one-to-many mapped to host AVX2 instructions. Therefore, this benchmark can fully utilize the host SIMD architecture and reach the greatest speed up among all selected benchmarks. We can find that ARMv7 achieves better performance than ARMv8. The reason is ARMv8 has 32 SIMD registers and the host, ie, AVX2, only has 16 SIMD registers. ARMv8 guest binaries using more than 16 registers may result in host

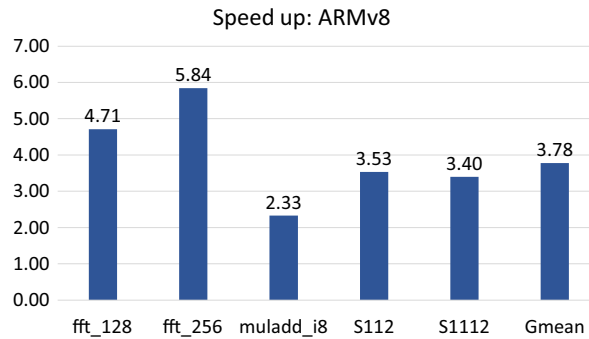


FIGURE 17 ARMv8 speed up [Colour figure can be viewed at wileyonlinelibrary.com]

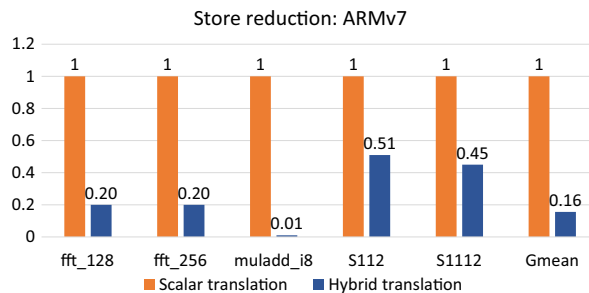


FIGURE 18 ARMv7 store reduction [Colour figure can be viewed at wileyonlinelibrary.com]

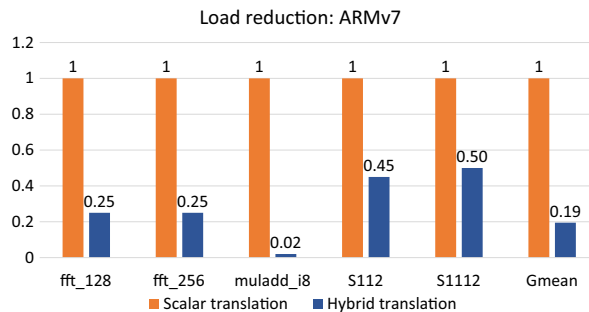


FIGURE 19 ARMv7 load reduction [Colour figure can be viewed at wileyonlinelibrary.com]

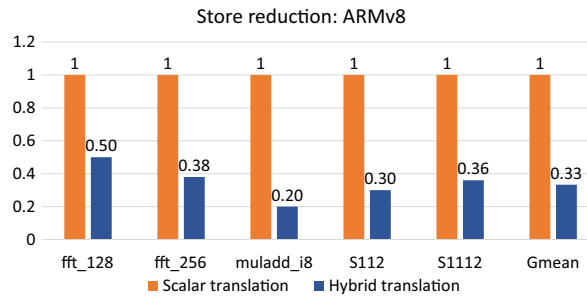


FIGURE 20 ARMv8 store reduction [Colour figure can be viewed at wileyonlinelibrary.com]

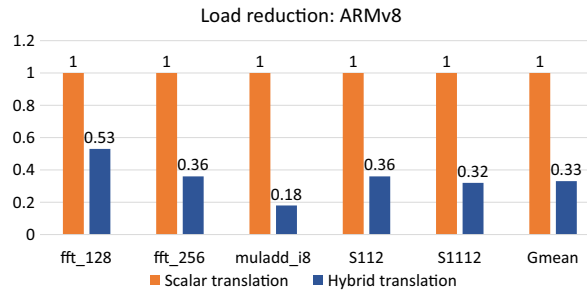


FIGURE 21 ARMv8 load reduction [Colour figure can be viewed at wileyonlinelibrary.com]

register spilling. Therefore, the reduction of L1 data cache access of ARMv7 guest is more significant than ARMv8 guest (shown in Figures 18 to 21).

4.3.2 | Multirix_Mul_i8

Most SIMD instructions in this benchmark are 8-bit multiply-and-accumulate and broadcast operations. However, X86-based machines do not support 8-bit multiply-and-accumulate operations so scalar-version HQEMU simulates 8-bit multiply-add instructions through helper functions. One multiply-and-accumulate instruction is translated to 4 helper functions. These helper function calls result in numerous load/store instructions for synchronizing the guest CPU state. By leveraging our helper function inlining mechanism, these load/store instructions can be eliminated. Therefore, L1 data cache loads/stores are significantly reduced.

We can find that L1 data cache loads/stores of ARMv7 are fewer than ARMv8. The reason is QEMU TCG uses different translation approaches to translate broadcast operations. When the guest is ARMv8, TCG translates broadcast operations into a sequence of 8-bit scalar loads/stores, which limit the optimizations of LLVM. When the guest is ARMv7, TCG translates broadcast operations into a sequence of 32-bit *shift* and *or* operations. These *shift* and *or* operations can be optimized by the LLVM optimizer and generate higher quality code. Although L1 data cache loads/stores of ARMv7 are fewer than ARMv8, their performance speed up is still close. This is because 8-bit multiply-and-accumulate operations are simulated by scalar instructions, dominating the execution time of this benchmark. As explained by Amdahl's law, the speed up coming from broadcast instructions translation is diluted.

4.3.3 | S112 and S1112

Most SIMD instructions in this benchmark are vector addition and *VTBL* (byte indexes table lookup),²⁵ which uses byte indexes to look up byte values in a list of SIMD registers. Because X86-based machines do not support the *VTBL* operation, scalar-version HQEMU translates *vtbl* into helper functions. Therefore, the major speedup comes from our selective helper function inlining scheme. On the other hand, ARMv8 *vtbl* supports 128-bit operands and ARMv7 *VTBL* only supports 64-bit operands. This results the number of *VTBL* instructions in ARMv7 executable is 2 times than that of ARMv8 executable. More *VTBL* instructions the executable has, more helper function inlining the translator performs. Typically, function inlining increases the code size and register pressure so the speedup of ARMv7 guest is not as good as ARMv8 guest.

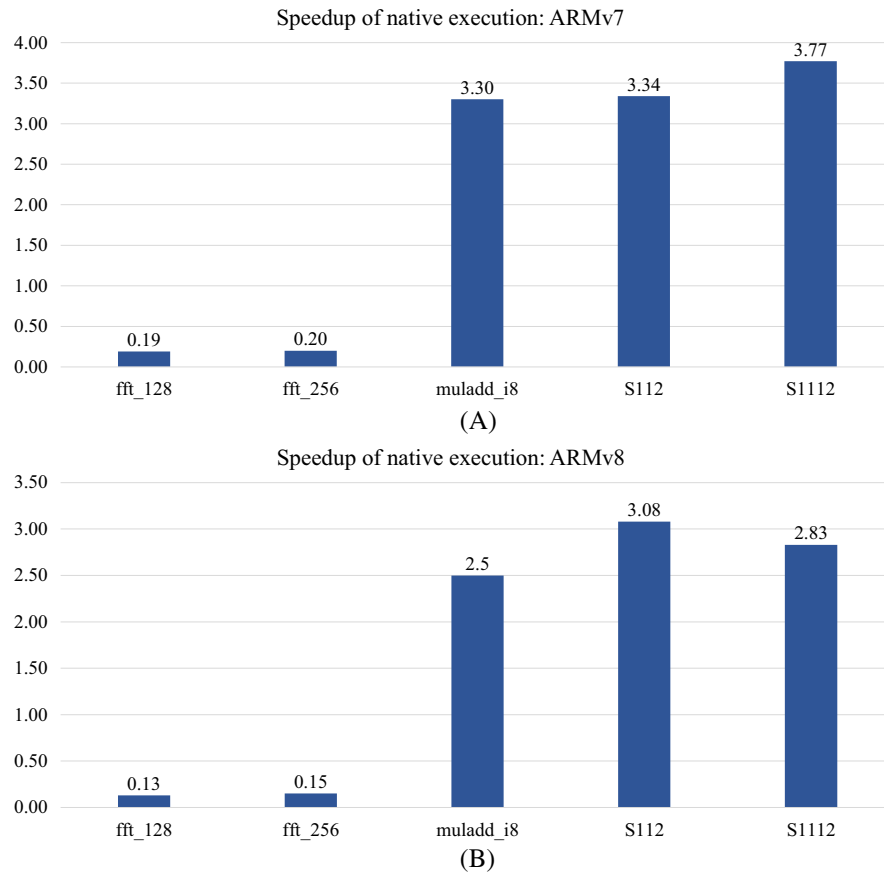


FIGURE 22 Compare with native execution. A, ARMv7 native execution; B, ARMv8 native execution [Colour figure can be viewed at wileyonlinelibrary.com]

4.3.4 | Compared with native execution

Figure 22 shows the comparison with native execution. To get native execution time, we run benchmarks on Odroid-C2 with Amlogic ARM Cortex-A53 in-order-execution 1.5 Ghz quadcore CPUs. The Odroid-C2 machine has 2 GB main memory and operating system is 64-bit Ubuntu-16.04 with Linux kernel 3.14.79. The numbers labeled in Figure 22 are $(\text{time of ARM} - \text{to} - \text{AVX2 translation}) / (\text{time of native execution})$. For example, 2.5 shown in mulladd_i8 of Figure 22B means the native execution runs 2.5 times faster than ARMv8-to-AVX2 translation. Our translator runs faster than native execution when benchmark is FFT. For example, ARMv8-to-AVX2 translation runs 7.69 times faster than native execution when benchmark is fft_128. This is because the host machine (Intel i7-5930K) is more powerful than Odroid-C2 and most SIMD instructions in the hot trace are translated via vector IRs.

5 | RELATED WORK

Most modern processors are already equipped with SIMD instructions. Many of them are being given even more powerful SIMD architecture/microarchitecture designs. For example, Intel and AMD have evolved their SIMD extensions from SSE to AVX, and to AVX2 and AVX-512. The SSE supports only 128-bit SIMD register length and 2-operands per instruction. The AVX2 supports 256-bit register length and 3 operands per instruction. The AVX-512 further increases the register length to 512-bit. ARMv8 also extends the NEON instruction set to support double precision and further develops the scalable vector extension architecture.⁵ However, SIMD architectures diverge between different ISAs. There have been some works focused on how to generate SIMD instruction for various platforms.⁴¹ Nuzman et al proposed Vapor SIMD⁴² framework. Vapor SIMD compiles source files (.c files) into portable vectorized bytecode first. Then, the just-in-time compiles the bytecode to native code and executes it at runtime. Nuzman et al^{41,42} compiled source files to vector forms prior to execution time.

For binary level SIMD optimizations, Hallou et al proposed a X86 dynamic optimizer transforming scalar code to SIMD and short-SIMD (SSE) to long-SIMD (AVX).⁴³ Their optimizer uses MCSema⁴⁴ to lift X86 binary to LLVM IRs and transforms the IRs by Polly. Finally, the IRs are compiled to machine code on-the-fly by LLVM just-in-time. For optimizations of SIMD code in DBT, Li et al proposed a SIMD data type tracking algorithm to trace the type of data element in SIMD registers.⁴⁵ They focus on SIMD register mapping rather than general SIMD code translation. Their work can decrease the amount of data movement and was implemented in IA-32 execution layer.³⁰ For dynamic SIMD translation, Fu et al⁴⁶ extended the vector IR to QEMU TCG IR so that the DBT can generate SIMD instructions. However, because their work is built on QEMU and QEMU directly maps the guest registers to host memory, they do not support register mapping and helper function inline mechanisms. Another related work to dynamic SIMD translation is the work of Fu et al,⁴⁷ and this paper is its extension. Guo et al proposed a SIMD translation framework⁴⁸ for static binary translation but only supports ARMv7 guest. They directly translated input binary to LLVM IRs and thus, do not need to deal with helper function issues. Shuja et al proposed SIMDOM framework.⁴⁹ They translated the ARM intrinsic functions to X86 intrinsic functions instead of translating ARM binary to X86 binary. For multilevel IRs and retargetable DBT, there has been little research on SIMD translation and optimization.

6 | CONCLUSION

DBTs have been widely used in virtualization, runtime optimization, program analysis, computer security enforcement, and software/hardware codesigns. The DBT dynamically translates guest executables to native instructions on the host. However, speeding up the emulation by generating optimized code has been challenging since optimizations require longer translation time, which is part of the runtime. Hybrid QEMU, which is a 2-level IR DBT, improves this weakness by moving the optimization to another thread to hide the optimization overhead. Hybrid QEMU runs SPEC 2006 benchmarks 2.4 times to 4 times faster than the original QEMU for emulating IA-32 executables on X86-64 machines. However, just like QEMU, HQEMU has not considered SIMD instruction translation. Since SIMD instructions are increasingly more common in applications, translating them in DBT should not be ignored.

In this paper, we have enhanced HQEMU by designing and implementing an efficient and retargetable SIMD translation framework. Our framework combines advantages of vector IRs and helper functions. We introduced a new set of vector IRs in QEMU TCG. The SIMD instructions in the guest executable are translated into vector IRs, and HQEMU converts these vector IRs to LLVM vector IRs. Then, we leverage LLVMs retargetable backends to emit SIMD instructions on the host platform. We also include a selective helper function inline scheme to minimize guest register state synchronization overhead. Compared with the scalar version HQEMU, our framework has achieved remarkable speedup. For the SPEC 2006 CFP benchmark, our framework runs up to 1.84 times faster, with a geometric mean of 1.35 times faster. For the ARMv7 guest executables, SPEC 2006 CFP contains few SIMD instructions because the NEON does not support double-precision operations. We also select a set of real world benchmarks to evaluate the ARMv7 performance and our framework runs up to 6.81 times faster when the benchmark is FFT. The proposed SIMD translation framework and guest state synchronization optimizations have been successfully demonstrated on HQEMU with popular architectures (ARM and X86). We believe the proposed schemes could be generalized to other DBT tools.

ACKNOWLEDGEMENTS

The authors are grateful to Professor Nigel Horspool and reviewers for their detailed comments and suggestions. This research was supported in part by Ministry of Science and Technology, Taiwan under grant number MOST 105-2622-8-002-002 and MOST-106-2218-E-002-040.

ORCID

Sheng-Yu Fu  <http://orcid.org/0000-0002-0893-2430>

REFERENCES

1. Lomont C. Introduction to intel advanced vector extensions. Intel White Paper; 2011.
2. Zekri AS. Enhancing the matrix transpose operation using intel AVX instruction set extension. *Int J Comput Sci Inf Technol*. 2014;6(3):67.

3. Jang M, Kim K, Kim K. The performance analysis of ARM NEON technology for mobile platforms. Paper presented at: Research in Applied Computation Symposium (RACS '11); 2011; Miami, FL. <http://doi.acm.org/10.1145/2103380.2103401>
4. Reddy VG. *Neon Technology Introduction*. Cambridge, UK: ARM Corporation; 2008.
5. ARM Ltd. *ARM Architecture Reference Manual Supplement*. Cambridge, UK: ARM Ltd; 2017;3.
6. Fuller S. Motorola's AltiVec technology. Vol 6. White Paper; 1998.
7. Nguyen H, John LK. Exploiting SIMD parallelism in DSP and multimedia algorithms using the AltiVec technology. Paper presented at: Proceedings of the 13th International Conference on Supercomputing; 1999; Rhodes, Greece. <http://doi.acm.org/10.1145/305138.305150>
8. MIPS architecture for programmers volume IV-j: The MIPS32 SIMD architecture. <https://imagination-technologies-cloudfront-assets.s3.amazonaws.com/documentation/MD00866-2B-MSA32-AFP-01.12.pdf>
9. Larsen S, Amarasinghe SP. Exploiting superword level parallelism with multimedia instruction sets. Paper presented at: Proceedings of the 2000 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI); 2000; Vancouver, Canada. <http://doi.acm.org/10.1145/349299.349320>
10. Porpodas V, Magni A, Jones TM. PSLP: padded SLP automatic vectorization. Paper presented at: Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization, (CGO); 2015; San Francisco, CA. <https://doi.org/10.1109/CGO.2015.7054199>
11. Rosen I, Nuzman D, Zaks A. Loop-aware SLP in GCC. Paper presented at: 2007 GCC Developers Summit; 2007; Ottawa, Canada.
12. Trifunovic K, Nuzman D, Cohen A, Zaks A, Rosen I. Polyhedral-model guided loop-nest auto-vectorization. Paper presented at: Proceedings of the 18th International Conference on Parallel Architectures and Compilation Techniques (PACT 2009); 2009; Raleigh, NC. <https://doi.org/10.1109/PACT.2009.18>
13. Karrenberg R, Hack S. Whole-function vectorization. Paper presented at: Proceedings of the 9th International Symposium on Code Generation and Optimization (CGO 2011); 2011; Chamonix, France. <https://doi.org/10.1109/CGO.2011.5764682>
14. Nuzman D, Zaks A. Outer-loop vectorization: revisited for short SIMD architectures. Paper presented at: 17th International Conference on Parallel Architecture and Compilation Techniques (PACT 2008); 2008; Toronto, Canada. <http://doi.acm.org/10.1145/1454115.1454119>
15. Nuzman D, Rosen I, Zaks A. Auto-vectorization of interleaved data for SIMD. Paper presented at: Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation; 2006; Ottawa, Canada. <http://doi.acm.org/10.1145/1133981.1133997>
16. Anderson A, Malik A, Gregg D. Automatic vectorization of interleaved data revisited. *TACO*. 2016;12(4). <http://doi.acm.org/10.1145/2838735>
17. Kim S, Han H. Efficient SIMD code generation for irregular kernels. *ACM Sigplan Notices*. 2012;47(8):55-64.
18. Zhou H, Xue J. Exploiting mixed SIMD parallelism by reducing data reorganization overhead. Paper presented at: Proceedings of the 2016 International Symposium on Code Generation and Optimization (CGO); 2016; Barcelona, Spain. <http://doi.acm.org/10.1145/2854038.2854054>
19. Sui Y, Fan X, Zhou H, Xue J. Loop-oriented array-and field-sensitive pointer analysis for automatic SIMD vectorization. Paper presented at: Proceedings of the 17th ACM SIGPLAN/SIGBED Conference on Languages, Compilers, Tools, and Theory for Embedded Systems; 2016; Santa Barbara, CA.
20. Zhou H, Xue J. A compiler approach for exploiting partial SIMD parallelism. *TACO*. 2016;13(1). <http://doi.acm.org/10.1145/2886101>
21. Ren G, Wu P, Padua DA. Optimizing data permutations for SIMD devices. Paper presented at: Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation; 2006; Ottawa, Canada. <http://doi.acm.org/10.1145/1133981.1133996>
22. Wu P, Eichenberger AE, Wang A. Efficient SIMD code generation for runtime alignment and length conversion. Paper presented at: 3rd IEEE/ACM International Symposium on Code Generation and Optimization (CGO); 2005; San Jose, CA. <https://doi.org/10.1109/CGO.2005.18>
23. Eichenberger AE, Wu P, O'Brien K. Vectorization for SIMD architectures with alignment constraints. Paper presented at: Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation; 2004; Washington, DC. <http://doi.acm.org/10.1145/996841.996853>
24. Bellard F. QEMU, a fast and portable dynamic translator. Paper presented at: 2005 USENIX Annual Technical Conference; 2005; Anaheim, CA. <http://www.usenix.org/events/usenix05/tech/freenix/bellard.html>
25. ARM Ltd. *ARM Architecture Reference Manual*. Cambridge, UK: ARM Ltd; 2014.
26. Hong D, Hsu C, Yew P, et al. HQEMU: A multi-threaded and retargetable dynamic binary translator on multicores. Paper presented at: 10th Annual IEEE/ACM International Symposium on Code Generation and Optimization (CGO); 2012; San Jose, CA. <http://doi.acm.org/10.1145/2259016.2259030>
27. Ebcioğlu K, Altman ER, Gschwind M, Sathaye SW. Dynamic binary translation and optimization. *IEEE Trans Comput*. 2001;50(6):529-548. <https://doi.org/10.1109/12.931892>
28. Smith JE, Nair R. *Virtual Machines: Versatile Platforms for Systems and Processes*. San Francisco, CA: Elsevier; 2005.
29. Zheng C, Thompson CL. PA-RISC to IA-64: transparent execution, no recompilation. *Computer*. 2000;33(3):47-52. <https://doi.org/10.1109/2.825695>
30. Baraz L, Devor T, Etzion O, et al. IA-32 execution layer: a two-phase dynamic translator designed to support IA-32 applications on Itanium-based systems. Paper presented at: Proceedings of the 36th Annual International Symposium on Microarchitecture. ACM/IEEE Computer Society; 2003; San Diego, CA. <https://doi.org/10.1109/MICRO.2003.1253195>
31. Kotha A, Anand K, Smithson M, Yellareddy G, Barua R. Automatic parallelization in a binary rewriter. Paper presented at: 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO); 2010; Atlanta, Georgia. <https://doi.org/10.1109/MICRO.2010.27>
32. Hong D, Fu S, Liu Y, Wu J, Hsu W. Exploiting longer SIMD lanes in dynamic binary translation. Paper presented at: 22nd IEEE International Conference on Parallel and Distributed Systems (ICPADS); 2016; Wuhan, China. <https://doi.org/10.1109/ICPADS.2016.0115>

33. Chow F. Intermediate representation. *ACM Queue*. 2013;11(10):30. <http://doi.acm.org/10.1145/2542661.2544374>
34. Merrill J. Generic and gimple: A new tree representation for entire functions. Paper presented at: Proceedings of the 2003 GCC Developers Summit; 2003; Ottawa, Canada.
35. Click C, Paleczny M. A simple graph-based intermediate representation. Paper presented at: Proceedings ACM SIGPLAN Workshop on Intermediate Representations (IR'95); 1995; San Francisco, CA. <http://doi.acm.org/10.1145/202529.202534>
36. Duboscq G, Würthinger T, Stadler L, Wimmer C, Simon D, Mössenböck H. An intermediate representation for speculative optimizations in a dynamic compiler. Paper presented at: Proceedings of the 7th ACM workshop on Virtual machines and intermediate languages; 2013; Indianapolis, IN.
37. Baghsorkhi SS, Vasudevan N, Wu Y. FlexVec: Auto-vectorization for irregular loops. Paper presented at: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI); 2016; Santa Barbara, CA. <http://doi.acm.org/10.1145/2908080.2908111>
38. Smith J. Future superscalar processors based on instruction compounding. http://www.ce.ewi.tudelft.nl/fileadmin/ce/homepages/galuzzi/File_index/Symposium/14_Smith.pdf
39. Puschel M, Moura JM, Johnson JR, et al. SPIRAL: code generation for DSP transforms. *Proc IEEE*. 2005;93(2):232-275.
40. Callahan D, Dongarra J, Levine D. Vectorizing compilers: a test suite and results. Paper presented at: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing (Supercomputing '88); 1988; Orlando, FL. <https://doi.org/10.1109/SUPERC.1988.44642>
41. Nuzman D, Henderson R. Multi-platform auto-vectorization. Paper presented at: 4th IEEE/ACM International Symposium on Code Generation and Optimization (CGO); 2006; New York, NY. <https://doi.org/10.1109/CGO.2006.25>
42. Nuzman D, Dyshel S, Rohou E, et al. Vapor SIMD: auto-vectorize once, run everywhere. Paper presented at: Proceedings of the 9th International Symposium on Code Generation and Optimization (CGO 2011); 2011; Chamonix, France. <https://doi.org/10.1109/CGO.2011.5764683>
43. Hallou N, Rohou E, Clauss P. Runtime vectorization transformations of binary code. *Int J Parallel Prog*. 2017;45(6):1536-1565. <https://doi.org/10.1007/s10766-016-0480-z>
44. McSema: x86 to machine code translation framework. <https://github.com/trailofbits/mcsema>
45. Li J, Zhang Q, Xu S, Huang B. Optimizing dynamic binary translation for SIMD instructions. Paper presented at: 4th IEEE/ACM International Symposium on Code Generation and Optimization (CGO); 2006; New York, NY. <https://doi.org/10.1109/CGO.2006.27>
46. Fu S, Wu J, Hsu W. Improving SIMD code generation in QEMU. Paper presented at: Proceedings of the 2015 Design, Automation and Test in Europe Conference and Exhibition; 2015; Grenoble, France. <http://dl.acm.org/citation.cfm?id=2757098>
47. Fu S, Hong D, Wu J, Liu P, Hsu W. SIMD code translation in an enhanced HQEMU. Paper presented at: 21st IEEE International Conference on Parallel and Distributed Systems (ICPADS); 2015; Melbourne, Australia. <https://doi.org/10.1109/ICPADS.2015.70>
48. Guo Y-C, Yang W, Chen J-Y, Lee J-K. Translating the ARM Neon and VFP instructions in a binary translator. *Softw Pract Exp*. 2016.
49. Shuja J, Gani A, Ko K, et al. SIMDOM: a framework for SIMD instruction translation and offloading in heterogeneous mobile architectures. *Trans Emerg Tel Tech*; 2017.

How to cite this article: Fu S-Y, Hong D-Y, Liu Y-P, Wu J-J, Hsu W-C. Efficient and retargetable SIMD translation in a dynamic binary translator. *Softw Pract Exper*. 2018;1-19. <https://doi.org/10.1002/spe.2573>